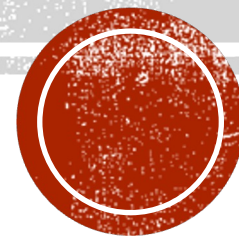


ELEMENTI DI PROGRAMMAZIONE NEI S.O.

Danilo Croce

Novembre 2025



IL MONDO SECONDO IL LINGUAGGIO C

- C è stato creato da Dennis Ritchie nel 1972 per sviluppare programmi UNIX.
- Alcune delle caratteristiche originali di UNIX sono ancora visibili
- “Everything is a file”



UNIX: EVERYTHING IS A FILE

- Sockets
- Devices
- Hard drives
- Stampanti
- Modems
- Pipes
- ...



C: EVERYTHING IS A FILE

Descriptive Name	Short Name	File Number	Description
Standard In	stdin	0	Input from the keyboard
Standard Out	stdout	1	Output to the console
Standard Error	stderr	2	Error output to the console

Per impostazione predefinita ogni processo inizia con questi 3 "file"... "aperti"



HELLO WORLD? (1 OF 3)

Cosa dobbiamo fare per stampare "Hello World" sulla console (output standard)?

- `printf(char *str, ...);`



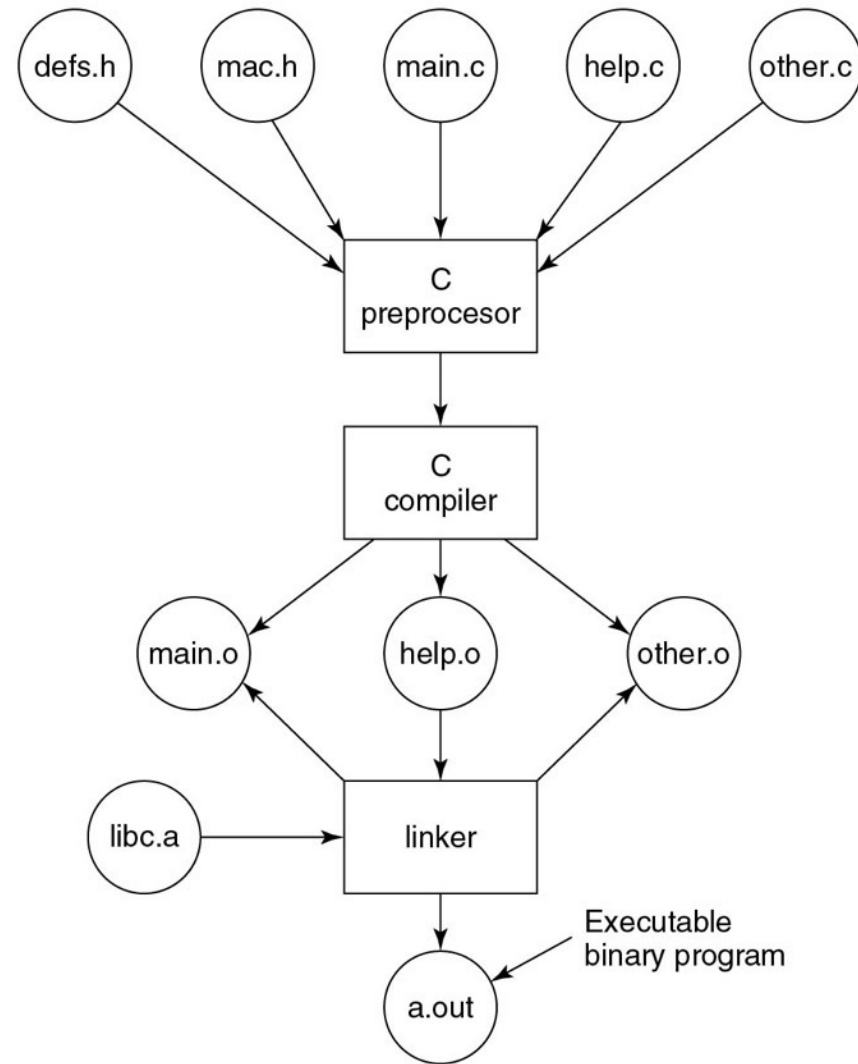
HELLO WORLD! (1 OF 3)

```
#include <stdio.h>

int main(int argc, char **argv)
{
    printf("Hello World!\n");
    return 0;
}
```



BUILD PROCESS



Il processo di compilazione dei file C e degli header file per creare un programma binario eseguibile.



HELLO WORLD? (2 OF 3)

Cosa dobbiamo fare per stampare "Hello World" sulla console (output standard)?

- `printf(char *str, ...);`
- `int write(int fd, char *buf, size_t len);`



IN EFFETTI: EVERYTHING IS A FILE!

Per impostazione predefinita ogni processo inizia con 3 "file" aperti

Descriptive Name	Short Name	File Number	Description
Standard In	stdin	0	Input from the keyboard
Standard Out	stdout	1	Output to the console
Standard Error	stderr	2	Error output to the console



HELLO WORLD! (2 OF 3)

```
#include <unistd.h>
#define STDOUT 1

int main(int argc, char **argv)
{
    char msg[] = "Hello World!\n";
    write(STDOUT, msg, sizeof(msg));
    return 0;
}
```



HELLO WORLD? (3 OF 3)

Cosa dobbiamo fare per stampare "Hello World" sulla console (output standard)?

- `printf(char *str, ...);`
- `int write(int fd, char *buf, size_t len);`
- `int syscall(int number, ...);`



HELLO WORLD! (3 OF 3)

```
#include <unistd.h>
#include <stdio.h>
#include <sys/syscall.h>
#define STDOUT 1

int main(int argc, char **argv)
{
    char msg[] = "Hello World!\n";
    int nr = SYS_write;
    syscall(nr, STDOUT, msg, sizeof(msg));
    return 0;
}
```



ESEMPI

- Fare riferimento ai file di esempio
 - `5.1_hello_world_1.c`
 - `5.1_hello_world_2.c`
 - `5.1_hello_world_3.c`

Mostrati a lezione



STANDARD LIBRARY

- Libc fornisce utili wrapper intorno alle syscall
 - Ad esempio `write`, `read`, `exit`
- È necessario chiamare la `syscall` oppure l'istruzione `int 0x80`
 - Fatto in assembly

```
syscall(int nr, ...)
```



... TANTE SYSCALL

Se fate click su Entry Point
finirete nel GITHUB di Linus Torvalds

<https://filippo.io/linux-syscall-table/>

The **return value** is placed in `%rax`.

%rax	Name	Manual	Entry point
0	read	read(2)	sys_read
%rdi unsigned int fd		%rsi char *buf	%rdx size_t count
1	write	write(2)	sys_write
2	open	open(2)	sys_open
3	close	close(2)	sys_close
%rdi unsigned int fd			
4	stat	stat(2)	sys_newstat
5	fstat	fstat(2)	sys_newfstat
6	lstat	lstat(2)	sys_newlstat
7	poll	poll(2)	sys_poll



UNA LUNGA ATTRAVERSATA... DA `read()` A `sys_read()`

- Cosa succede quando viene invocata la funzione `read()`?
 - Di seguito viene riportata la sequenza dei passaggi effettivi.
 - *Disclaimer:* alcuni punti sono semplificati ma rappresentano correttamente la catena di chiamate reali.

LET'S START!

La funzione `read()` è definita come alias di `__libc_read`, che funge da wrapper per la chiamata di sistema `read`.

- **File:** `glibc/sysdeps/unix/sysv/linux/read.c`

- **Contenuto rilevante:**

```
ssize_t
__libc_read (int fd, void *buf, size_t nbytes)
{
    return SYSCALL_CANCEL (read, fd, buf, nbytes);
}
```

- **Link:**

<https://codebrowser.dev/glibc/glibc/sysdeps/unix/sysv/linux/read.c.html>



UNA LUNGA ATTRAVERSATA... DA `read()` A `sys_read()`

La macro `SYSCALL_CANCEL` è definita per rendere la chiamata di sistema compatibile con la cancellazione dei thread (pthread cancellation).

- In particolare, abilita e disabilita la cancellazione asincrona attorno all'invocazione della syscall.
- All'interno, chiama la macro `INLINE_SYSCALL_CALL(read, fd, buf, nbytes)`.
 - **File:** `glibc/sysdeps/unix/sysdep.h`
 - **Link:** <https://codebrowser.dev/glibc/glibc/sysdeps/unix/sysdep.h.html>
- **Funzione pratica:** permette a `glibc` di gestire i thread senza rischiare corruzione di stato quando una chiamata di sistema viene interrotta o cancellata.



UNA LUNGA ATTRAVERSATA... DA `read()` A `sys_read()`

3. Macro `INLINE_SYSCALL_CALL` / `INTERNAL_SYSCALL`: La macro `INLINE_SYSCALL_CALL` usa `INTERNAL_SYSCALL` per costruire l'istruzione assembly che effettua la system call.

- `INTERNAL_SYSCALL` è definita in modo architettura-specifico, per `x86_64` si trova in:
 - **File:** `glibc/sysdeps/unix/sysv/linux/x86_64/sysdep.h`
 - **Link:**
https://codebrowser.dev/glibc/glibc/sysdeps/unix/sysv/linux/x86_64/sysdep.h.html
- Questa macro prepara i registri con i parametri della system call secondo la convenzione `x86_64`:
 - `%rdi` = primo argomento (`fd`)
 - `%rsi` = secondo argomento (`buf`)
 - `%rdx` = terzo argomento (`count`)
 - `%eax` = numero della system call (`__NR_read`, cioè 0)
- Esegue poi l'istruzione `syscall`, che provoca il salto in modalità kernel.
- In sostanza, questo è **il punto esatto dove viene invocata l'istruzione `syscall`** all'interno di `glibc`.



UNA LUNGA ATTRAVERSATA... DA `read()` A `sys_read()`

- La macro `internal_syscall3`, derivata da `INTERNAL_SYSCALL`, è quella effettivamente usata per invocare `read()` (che ha tre argomenti: `fd`, `buf`, `count`):

```
#define internal_syscall3(number, arg1, arg2, arg3) \
({ \
    unsigned long resultvar; \
    register long _a1 asm("rdi") = arg1; \
    register long _a2 asm("rsi") = arg2; \
    register long _a3 asm("rdx") = arg3; \
    asm volatile ("syscall\n\t" \
        : "=a"(resultvar) \
        : "0"(number), "r"(_a1), "r"(_a2), "r"(_a3) \
        : "memory", "cc", "r11", "cx"); \
    (long) resultvar; \
})
```

- È qui che viene realmente eseguito **syscall**: la macro carica il numero della system call in **%rax** e passa gli argomenti nei registri appropriati secondo la convenzione Linux a 64 bit.
- Il parametro **number** è una **costante definita dal kernel** e resa disponibile da glibc tramite gli header:
 - https://github.com/torvalds/linux/blob/master/arch/x86/entry/syscalls/syscall_64.tbl
 - Esempio: 0 common read sys_read



UNA LUNGA ATTRAVERSATA... DA `read()` A `sys_read()`

Istruzione `syscall`: passaggio in modalità kernel

- L'istruzione `syscall` è un'istruzione del processore x86_64 che serve a passare dal contesto utente a quello kernel.
- Quando viene eseguita:
 - La CPU salva lo stato corrente del processo.
 - Cambia lo stack e i segmenti per entrare nello spazio kernel.
- Trasferisce il controllo all'indirizzo definito nel registro `MSR_LSTAR`, che in Linux corrisponde all'etichetta `entry_SYSCALL_64`.
 - **File (kernel):** `arch/x86/entry/entry_64.S`
 - **Link:**
https://github.com/torvalds/linux/blob/master/arch/x86/entry/entry_64.S
- Questa è la prima istruzione eseguita nel kernel come parte della chiamata di sistema `read()`



UNA LUNGA ATTRAVERSATA... DA `read()` A `sys_read()`

- Nel file `entry_64.S`, il simbolo `entry_SYSCALL_64` rappresenta l'entry point comune per tutte le system call a 64 bit.
- Da qui inizia la parte “kernel” della chiamata:

```
SYM_CODE_START(entry_SYSCALL_64)
    UNWIND_HINT_ENTRY
    ENDBR
    swapgs                # scambia GS base (user → kernel)
    movq %rsp, PER_CPU_VAR(cpu_tss_rw + TSS_sp2) # salva lo stack utente
    SWITCH_TO_KERNEL_CR3 scratch_reg=%rsp        # cambia page table (user→kernel)
    movq PER_CPU_VAR(cpu_current_top_of_stack), %rsp # imposta stack del kernel
```

- `swapgs`: cambia il puntatore GS per usare la struttura per-CPU del kernel.
- `movq %rsp, TSS_sp2`: salva lo stack utente nella TSS come riferimento temporaneo.
- `SWITCH_TO_KERNEL_CR3`: passa alla **page table del kernel** (protezione memoria).
- `movq cpu_current_top_of_stack, %rsp`: imposta lo **stack del kernel** corrente.
- Dopo questo setup, il kernel può continuare con `do_syscall_64()`



UNA LUNGA ATTRAVERSATA... DA `read()` A `sys_read()`

- `do_syscall_64()` è una **funzione C del kernel Linux** che rappresenta il vero **dispatcher** delle chiamate di sistema a 64 bit.
 - È chiamata direttamente da `entry_SYSCALL_64` (in assembly) dopo il passaggio in modalità kernel.
- Riceve:
 - il puntatore alla struttura dei registri (`struct pt_regs *regs`)
 - il numero della system call (`nr`, ad esempio `__NR_read = 0`)
- Esegue il flusso principale:
 - `syscall_enter_from_user_mode()`: gestisce il contesto di ingresso nel kernel.
 - `do_syscall_x64()`: sceglie la funzione appropriata nella `sys_call_table`.
 - `syscall_exit_to_user_mode()`: prepara il ritorno allo user space.
- **Link:** https://github.com/torvalds/linux/blob/master/arch/x86/entry/syscall_64.c



UNA LUNGA ATTRAVERSATA... DA

`read()` A `sys_read()`

- `do_syscall_64()` chiama `do_syscall_x64()`, che:
 - Controlla che il numero della system call (`nr`) sia valido.
 - Se valido, richiama la funzione `x64_sys_call()`.
- `x64_sys_call()` contiene uno **switch** generato automaticamente da `asm/syscalls_64.h`:
 - Ogni caso (case `nr`) richiama la funzione corrispondente, ad esempio:
 - case 0 → `__x64_sys_read(regs)`
 - case 1 → `__x64_sys_write(regs)`
- Quindi, per `read()`, il kernel chiama `__x64_sys_read()`, definita nel file `fs/read_write.c`.
 - richiama la funzione `ksys_read(fd, buf, count)` che implementa la logica reale:
 - accede al file descriptor
 - legge i dati nel buffer kernel
 - copia i dati nello spazio utente.



DA DOVE VIENE `__x64_sys_read()` ?

- La system call `read()` è registrata nel file:

- https://github.com/torvalds/linux/blob/master/arch/x86/entry/syscalls/syscall_64.tbl

che è nella forma

```
0 common read sys_read
1 common write sys_write
```

- Lo script:

- <https://github.com/torvalds/linux/blob/master/scripts/syscalltbl.sh>

genera il file

- `arch/x86/include/generated/asm/syscalls_64.h`

contenente:

- `__SYSCALL(0, sys_read)`

- Questo viene incluso in:

- `arch/x86/entry/syscall_64.c`

che diventa:

- `case 0: return __x64_sys_read(regs);`

→ Lo “switch” che collega il numero 0 alla funzione `__x64_sys_read()`.



CHI CREA `__x64_sys_read()` ?

- In realtà non esiste nel sorgente come funzione esplicita. È **generata automaticamente** dalla macro `SYSCALL_DEFINE3()`.

- File: https://github.com/torvalds/linux/blob/master/fs/read_write.c

- **Contenuto:**

```
SYSCALL_DEFINE3(read, unsigned int, fd,
                 char __user *, buf,
                 size_t, count)
{
    return ksys_read(fd, buf, count);
}
```

- **La macro `SYSCALL_DEFINE3` (definita in `include/linux/syscalls.h`) espande in fase di compilazione:**

```
asmlinkage long __x64_sys_read(unsigned int fd,
                                char __user *buf,
                                size_t count)
{
    return ksys_read(fd, buf, count);
}
```



ESEMPI DI SYSCALL_DEFINE3 NEL KERNEL LINUX

- read: fs/read_write.c → ksys_read(fd, buf, count)
- write: fs/read_write.c → ksys_write(fd, buf, count)
- open: fs/open.c → do_sys_open(AT_FDCWD, filename, flags, mode)
- newfstatat: fs/stat.c → vfs_fstatat(dfd, filename, statbuf)
- msgget: ipc/msg.c → ksys_msgget(key, msgflg, cmd)
- mprotect: mm/mprotect.c → do_mprotect_pkey(start, len, prot, -1)
- setpriority: kernel/sys.c → set_user_nice(which, who, niceval)
- sched_setattr: kernel/sched/core.c → __sched_setattr(pid, uattr, flags)
- chmod: fs/open.c → do_fchmodat(AT_FDCWD, filename, mode)
- linkat: fs/namei.c → do_linkat.olddfd, oldname, newdfd, newname, flags)
- renameat: fs/namei.c → do_renameat2.olddfd, oldname, newdfd, newname, 0)
- utimensat: fs/utimes.c → do_utimes(dfd, filename, utimes, flags)

- **La lista completa su :**

<https://syscalls.mebeim.net/?table=x86/64/x64/latest>

▪



DALLA `read()` DI USER SPACE A `ksys_read()` NEL KERNEL

- **1. User space**
 - La funzione C `read(fd, buf, count)` invoca la system call `read`
 - Il numero di syscall (`__NR_read = 0`) viene caricato in `%rax`
- **2. Entry point in assembly**
 - File: `arch/x86/entry/entry_64.S`
 - L'istruzione `SYSCALL` causa il salto in modalità kernel
 - Il controllo passa a `entry_SYSCALL_64()`, poi a `do_syscall_64()`
- **3. Preparazione del contesto kernel**
 - Funzione: `syscall_enter_from_user_mode()`
 - Salva registri, verifica privilegi e prepara lo stack kernel
- **4. Dispatcher della system call**
 - File: `arch/x86/entry/syscall_64.c`
 - `do_syscall_64()` chiama `do_syscall_x64()`
 - Quest'ultima invoca `x64_sys_call(regs, nr)`
 - Switch-case generato automaticamente:
 - `→ case 0: return __x64_sys_read(regs);`



DALLA `read()` DI USER SPACE A `ksys_read()` NEL KERNEL

▪ 5. Definizione della syscall

- File: `fs/read_write.c`
- Macro: `SYSCALL_DEFINE3(read, unsigned int fd, char __user *buf, size_t count)`
- Implementa `__x64_sys_read()`, che chiama `ksys_read(fd, buf, count)`

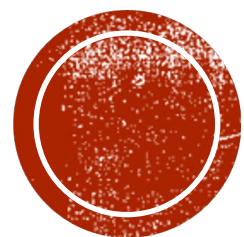
▪ 6. Implementazione kernel pura

- File: `fs/read_write.c`
- Funzione `ksys_read()` esegue la logica della read:
 - → lookup del file descriptor
 - → controllo dei permessi
 - → lettura tramite `vfs_read()`

▪ 7. File system layer

- `vfs_read()` chiama `file->f_op->read_iter`
- Questa arriva infine al driver o al filesystem (es. ext4, procfs, socket...)
- ... vedi prossime lezioni sul tema 😊





CREAZIONE DI PROCESSI



PROCESS MANAGEMENT SYSTEM CALL

- Creiamo una shell minimale:
 - Attende che l'utente digiti un comando
 - Avvia un processo per eseguire il comando
 - Attende che il processo sia terminato

`(fork, wait, execv)`



CREAZIONE DEL PROCESSO (1 OF 2)

- `pid_t fork()`
 - Duplica il processo corrente
 - Restituisce il pid del figlio nel chiamante (genitore)
 - Restituisce 0 nel nuovo processo (figlio)
- `pid_t wait(int *wstatus)`
 - Attende che i processi figli cambino stato
 - Scrive lo stato in `wstatus`
 - Ad esempio, a causa di un `exit` o segnale



FORK, WAIT

```
void main(void)
{
    int pid, child_status;
    if (fork() == 0) {
        do_something_in_child();
    } else {
        wait(&child_status); // Wait for child
    }
}
```



CREAZIONE DEL PROCESSO (2 OF 2)

- `int execlv(const char *path, char *constargv[ ] );`
 - Carica un nuovo binario (path) nel processo corrente, rimuovendo tutte le altre mappature di memoria.
 - `constargv` contiene gli argomenti del programma
 - L'ultimo argomento è `NULL`
 - E.g., `constargv = { "/bin/ls", "-a", NULL }`
 - Diverse varianti di `exec(v)(p)` (controllare le pagine `man`)



FORK, WAIT, EXEVCV

```
void main(void)
{
    int    pid, child_status;
    char    *args[] = {"/bin/ls", "-l", NULL};
    if (fork() == 0) {                // fork creates child process
        exevcv(args[0], args);        // in child: load+execute program
    } else {
        wait(&child_status);          // Wait for child
    }
}
```



UNA SHELL MINIMALE

```
while (1) {
    char cmd[256], *args[256];
    int status;
    pid_t pid;
    read_command(cmd, args); /* reads command and arguments from command line */

    pid = fork();

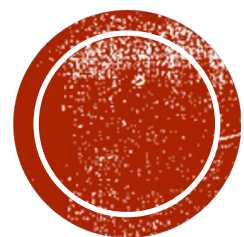
    if (pid == 0) {
        execv(cmd, args);
        exit(1);
    } else {
        wait(&status);
    }
}
```



COME TERMINARE I PROGRAMMI?

- Ctrl+C, ma come funziona?
- Risposta: i **segnali**





GESTIONE DEI SEGNALI



CHIAMATE DI SISTEMA PER I SEGNALI

- A volte i processi devono essere interrotti durante la loro esecuzione.
- Viene inviato un **segnale** al processo che deve essere interrotto.
- Il processo interrotto può catturare il segnale installando un gestore di segnali (**signal handler**)
- Cosa succede quando l'utente del terminale preme CTRL+C o CTRL+Z ?

(signal, alarm, kill)



SIGNAL, ALARM, KILL

- `sighandler_t signal(int signum, sighandler_t handler)`
 - **Registra un gestore di segnali per il segnale `signum`**
- `unsigned int alarm(unsigned int seconds)`
 - **Consegna `SIGALRM` in un numero di secondi specificato**
- `int kill(pid_t pid, int sig)`
 - **Consegna il segnale `sig` al processo `pid` (non uccide!!!)**



ALARM EXAMPLE

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void alarm_handler(int signal)
{
    printf("In signal handler: caught signal %d!\n", signal);
    exit(0);
}

int main(int argc, char **argv)
{
    signal(SIGALRM, alarm_handler);
    alarm(1); // alarm will send signal after 1 sec

    while (1) {
        printf("I am running!\n");
    }
    return 0;
}
```





COMUNICAZIONE TRA PROCESSI ATTRAVERSO PIPE



PIPE EXAMPLE (1 OF 2)

Cosa succede se si esegue il seguente comando?

```
$ cat names.txt | sort
```

E i seguenti comandi?

```
$ mkfifo named.pipe
```

```
$ echo "Hello World!" > named.pipe
```

```
$ cat named.pipe
```

(open, close, pipe, dup)



OPEN, CLOSE, PIPE, DUP

- `int open(const char *pathname, int flags)`
 - **Apre il file specificato dal nome del percorso (`pathname`)**
- `int close(int fd)`
 - **Chiude il descrittore di file specificato `fd`**
- `int pipe(int pipefd[2])`
 - **Crea una pipe con due `fd` per le sue estremità**
- `int dup(int oldfd)`
 - **Crea una copia del descrittore di file `oldfd` utilizzando il descrittore di file inutilizzato con il numero più basso per la copia.**



MA DUP A CHE SERVE?

- `dup` e `dup2` sono chiamate di sistema su Unix/Linux per duplicare descrittori di file.
- **Funzione:**
 - `dup(int oldfd)`: Crea un duplicato del descrittore `oldfd`.
 - `dup2(int oldfd, int newfd)`: Duplica `oldfd` su un descrittore specifico `newfd`.
- **Perché?:** è comune voler reindirizzare l'output di un programma verso un file o un altro programma. `dup` e `dup2` permettono di «agganciare» il descrittore di output standard (`STDOUT_FILENO`, ovvero 1) o di input standard (`STDIN_FILENO`, ovvero 0) a un file.
- **Utilità:** Permettono la gestione avanzata di input/output, come il reindirizzamento di file, la creazione di pipeline tra processi, e la gestione di input/output in processi figli.



ESEMPIO 1: REINDIRIZZAMENTO DI OUTPUT

- **Scenario:** Salvare l'output di un programma in un file.

- **Esempio:**

```
int file_fd = open("output.txt", O_WRONLY | O_CREAT, 0644);  
dup2(file_fd, STDOUT_FILENO); // Reindirizza stdout su file_fd  
printf("Questo va in output.txt\n");  
close(file_fd);
```

- **Risultato:** L'output viene scritto in output.txt invece che in console.
- **Utilizzo tipico:** Registrazione di log, salvataggio di output in file.



ESEMPIO 2: CREAZIONE DI PIPELINE TRA PROCESSI

- **Scenario:** Collegare l'output di un programma all'input di un altro

Esempio: `ps aux | grep httpd`

```
int fd[2];
pipe(fd);
if (fork() == 0) {
    close(fd[0]);
    dup2(fd[1], STDOUT_FILENO); // Output di `ps aux` nella pipe
    execlp("ps", "ps", "aux", NULL);
} else {
    close(fd[1]);
    dup2(fd[0], STDIN_FILENO); // Input di `grep` dalla pipe
    execlp("grep", "grep", "httpd", NULL);
}
```

- **Risultato:** `ps aux` manda l'output a `grep`, simulando una pipeline.
- **Utilizzo tipico:** Shell scripting, automazione di comandi.



ESEMPIO 3: GESTIONE DEI PROCESSI FIGLI

- **Scenario:** Reindirizzare l'output di un processo figlio a un file di log.

```
if (fork() == 0) {  
    int log_fd = open("logfile.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);  
    dup2(log_fd, STDOUT_FILENO); // Output va in logfile.txt  
    close(log_fd);  
    execlp("ls", "ls", "-l", NULL); // Comando eseguito dal figlio  
}
```

- **Risultato:** L'output del processo figlio `ls -l` va in `logfile.txt`
- **Nota:** `execlp` sostituisce il processo corrente con il processo specificata dal file
- **Utilizzo tipico:** Logging centralizzato, configurazione di ambienti di processo.



PIPE EXAMPLE (2 OF 2)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define STDIN 0
#define STDOUT 1

#define PIPE_RD 0
#define PIPE_WR 1

int main(int argc, char** argv)
{
    pid_t cat_pid, sort_pid;
    int fd[2];

    pipe(fd);
    cat_pid = fork();
    if ( cat_pid == 0 ) {
        close(fd[PIPE_RD]);
        close(STDOUT);
        dup(fd[PIPE_WR]);
        execl("/bin/cat", "cat", "names.txt" , NULL);
    }
}
```

```
sort_pid = fork();
if ( sort_pid == 0 ) {
    close(fd[PIPE_WR]);
    close(STDIN);
    dup(fd[PIPE_RD]);
    execl("/usr/bin/sort", "sort", NULL);
}

close(fd[PIPE_RD]);
close(fd[PIPE_WR]);

/* wait for children to finish */
waitpid(cat_pid, NULL, 0);
waitpid(sort_pid, NULL, 0);

return 0;
}
```



POSSIAMO SALTARE QUESTE “CHIUSURE”?

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define STDIN 0
#define STDOUT 1

#define PIPE_RD 0
#define PIPE_WR 1

int main(int argc, char** argv)
{
    pid_t cat_pid, sort_pid;
    int fd[2];

    pipe(fd);

    cat_pid = fork();
    if ( cat_pid == 0 ) {
        close(fd[PIPE_RD]); ←
        close(STDOUT);
        dup(fd[PIPE_WR]);
        execl("/bin/cat", "cat", "names.txt" , NULL);
    }
}
```

```
sort_pid = fork();
if ( sort_pid == 0 ) {
    close(fd[PIPE_WR]); ←
    close(STDIN);
    dup(fd[PIPE_RD]);
    execl("/usr/bin/sort", "sort", NULL);
}

close(fd[PIPE_RD]);
close(fd[PIPE_WR]);

/* wait for children to finish */
waitpid(cat_pid, NULL, 0);
waitpid(sort_pid, NULL, 0);

return 0;
}
```

And why / why not?



POSSIAMO SALTARE QUESTE “CHIUSURE”?

(RISPOSTA)

1. **Evitare Blocchi:** Chiudi la fine di lettura di una pipe per impedire al processo di scrittura di rimanere bloccato.
2. **Ricezione EOF:** sort aspetta un EOF per terminare la lettura. Chiudi la fine di scrittura per inviare un EOF a sort.
3. **Evitare Letture Accidentali:** Nel processo cat, chiudi la fine di lettura per evitare letture inaspettate dalla pipe.
4. **Reindirizzamento di STDOUT in cat:** Dopo la duplicazione, chiudi il descriptor originale per garantire che cat scriva solo nella pipe.
5. **Reindirizzamento di STDIN in sort:** Dopo la duplicazione, chiudi il descriptor originale per assicurarti che sort legga solo dalla pipe.
6. **Descriptor nel Processo Padre:** Dopo la fork, il processo padre dovrebbe chiudere entrambe le estremità della pipe.



ESEMPI

- **Creazione di processi paralleli con metodo fork**
- `5.2_my_first_fork_1.c`
 - **un semplice esempio di invocazione del metodo fork**
- `5.2_my_first_fork_2.c`
 - **creazione dei processi e avvio di applicazioni da codice C**
- `5.3_my_signal_1.c`
 - **Gestione dei segnali**
- `5.3_my_signal_2.c`
 - **Gestione degli allarmi**



ESEMPI (CONT)

- `5.4_fork_pipe.c`
 - **Comunicazione tra processi attraverso PIPE**
- `5.5_my_first_bash.c`
 - **Sviluppo di una BASH minimale in C**
- **NON sottovalutate lo script usato per compilare gli esempi**



ESERCIZIO

- Un processo genera due processi figli P1 e P2.
- Il figlio P1 esegue un ciclo indeterminato durante il quale genera casualmente numeri interi compresi tra 0 e 100.
 - P1 comunica, ad ogni iterazione, il numero al padre solo se esso è dispari.
- P2 fa la stessa cosa ma comunica al padre solo i numeri pari.
- Il padre, per ogni coppia di numeri che riceve dai figli ne fa la somma e la visualizza.
- Il programma deve terminare quando la somma dei due numeri ricevuti supera il valore 190: il padre, allora, invia un segnale di terminazione a ciascuno dei figli.

